

Aggregation vs Composition

Aggregation (Has-A / Weak)

Objects have their own lifecycle. The child can exist independently of the parent container.

- **Hotel & Rooms:** If the Hotel closed down, Rooms could theoretically be converted or managed separately.
- **Dynamic association:** The lifecycle of rooms is not strictly bound to one parent entity.

Composition (Part-Of / Strong)

The child object's life cycle is completely dependent on the parent. It cannot exist on its own.

- **Booking System:** A booking detail cannot exist without an active booking transaction.
- **Strict dependency:** Deleting the primary Booking automatically destroys all associated system logs.

Two Ways Objects Can Be Related

Inheritance (IS-A / Strong)

A child class inherits behavior and attributes from another parent class, establishing a formal taxonomy.

- **Dog IS-A Animal:** The Dog subclass automatically inherits the base traits defined in Animal.
- **Structured Hierarchy:** Also models relationships like Teacher IS-A Person or Car IS-A Vehicle.

Python Code Implementation

The subclass passes the parent class name inside parentheses to establish the inheritance relationship.

```
class Animal:  
    pass  
  
class Dog(Animal):  
    pass
```

Aggregation (HAS-A)

Concept & Examples

One object contains another object as part of its state, representing a weak association where lifecycles remain independent.

- **Hotel HAS Rooms:** Rooms can exist independently even if the hotel organization structure changes.
- **Booking HAS Customer:** A customer profile persists in the database regardless of active bookings.
- **Library HAS Books:** Books are individual entities that can exist outside a specific library system.

Python Implementation

Container classes maintain references to target instances dynamically, often stored inside collection attributes.

```
class Hotel:
    def __init__(self):
        self.rooms = []

# The list holds dynamic references to Room objects,
decoupling their lifetime from the Hotel instance.
```

Inheritance (IS-A)

Concept & Hierarchy

A subclass inherits attributes and methods from a parent class, establishing a formal taxonomy and promoting code reusability.

Animal (Parent Class)



Dog (Subclass / Child)

Python Implementation

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def bark(self):
        print("Woof!")
```

```
dog = Dog()
dog.speak() # Some sound
dog.bark() # Woof!
```

Console Output:

```
Some sound
Woof!
```

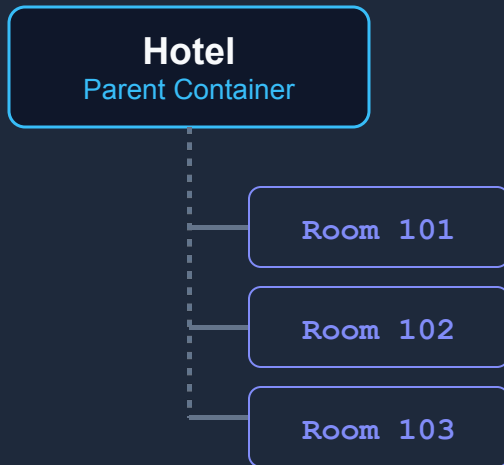
Aggregation (HAS-A Relationship)

Dynamic Lifecycle

In an Aggregation relationship, the child objects are not bound strictly to the lifetime of the parent container.

A Hotel has many Rooms: This represents a weak association. The Hotel acts as the organizational shell, while individual rooms hold their own distinct status.

Independent Existence: If the Hotel object is destroyed or removed from the system, the Room objects continue to exist independently in memory or storage.



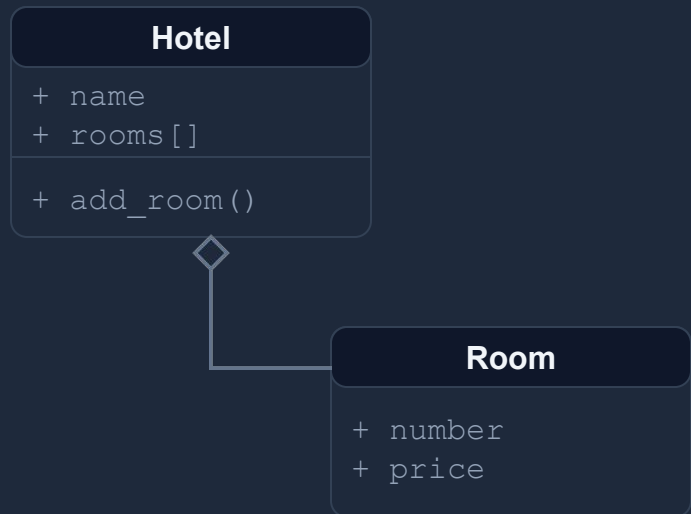
Aggregation UML Diagram

Visualizing the Model

UML aggregation uses an open, white diamond pointing to the container class ([Hotel](#)) to denote a "HAS-A" relationship with independent lifetimes.

Hotel (Container): Holds a collection of Room references without owning their lifecycle.

Room (Element): Exists independently even if the parent Hotel object is destroyed.



◇ White diamond represents Aggregation (HAS-A)

Python Translation

Room Class

Defines individual components with independent state and attributes.

```
class Room:
    def __init__(self, number, price):
        self.number = number
        self.price = price
```

Each Room instance maintains its own attributes, decoupled from any container organization.

Hotel Class

Maintains dynamic references to instances without managing their lifecycles.

```
class Hotel:
    def __init__(self, name):
        self.name = name
        self.rooms = []

    def add_room(self, room):
        self.rooms.append(room)
```

The rooms list holds outer references, completing the weak HAS-A relationship model.

Creating Objects & Memory

Instantiating & Linking

In aggregation, child components are instantiated independently of their container, then passed in as external references.

```
room1 = Room(101, 120)
room2 = Room(102, 150)
hotel = Hotel("Sunshine Hotel")

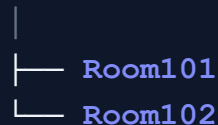
hotel.add_room(room1)
hotel.add_room(room2)
```

Notice: The hotel does NOT create the rooms; they are created externally and stored as references.

Memory Map

Object Hierarchy

Hotel



Displaying the Rooms

Iteration & Access

To access aggregated objects, we simply iterate over the container's list holding references to the independent instances.

```
for room in hotel.rooms:  
    print(room.number, room.price)
```

No ownership: Aggregation simply stores references to other objects. Deleting the `hotel` object does not delete the `room` instances, as they are not created inside the class.

Terminal Output

Console Exec

output

101 120

102 150

Introducing Booking

Multi-Object Aggregation

A Booking consolidates the workflow by aggregating both Room and Customer instances, representing a dynamic HAS-A relationship with independent lifetimes.

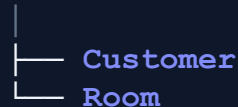
```
class Booking:
    def __init__(self, customer, room):
        self.customer = customer
        self.room = room
```

Dynamic Links: Both objects exist independently before the booking is created and survive if the booking is cancelled.

Memory Map

Booking Relationships

Booking



◇ Booking holds references to independent components

Booking UML Diagram

UML Relationship

The Booking class acts as an aggregation container. Both Customer and Room are independent entities created beforehand and passed into the Booking constructor as references.

Since it models an [Aggregation \(HAS-A\)](#) relationship, destroying a Booking instance does not destroy its associated Customer or Room instances.

UML Schema

Class Definition

Booking

```
- booking_id
- customer
- room
```

◇ Lifetimes of Customer and Room are independent of Booking

Implementing the Classes

Class Architecture

We establish independent blueprints for both our entities. The Booking constructor accepts independent Customer and Room references, implementing the aggregation logic cleanly.

```
class Customer:
    def __init__(self, name):
        self.name = name

class Booking:
    def __init__(self, booking_id, customer, room):
        self.booking_id = booking_id
        self.customer = customer
        self.room = room
```

Lifetime Independence: Neither customer nor room is bound to the booking's lifecycle. They survive booking destruction.

Object State

Composition Flow

Booking

- **booking_id**
- **customer**
- **room**

◇ Properties hold external object memory references

Creating a Booking

Instantiation Workflow

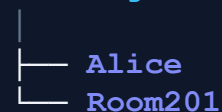
We instantiate independent entities first, then pass their memory references into the Booking constructor to establish the aggregation.

```
customer = Customer("Alice")  
room = Room(201, 180)  
booking = Booking("B001", customer, room)
```

Direct Reference: The resulting booking object holds active references to the pre-existing Alice and Room201 instances, showcasing Python's object linking capability.

Active Graph Runtime Relations

Booking



◇ References map directly to memory addresses of argument objects

Complete Example

Hotel & Booking Setup

We instantiate a Hotel, populate it with Room entities, create a Customer, and tie them together within a transient Booking instance.

```
hotel = Hotel("Paradise Hotel")
room1 = Room(101, 100)
room2 = Room(102, 150)
hotel.add_room(room1)
hotel.add_room(room2)
customer = Customer("John")
booking = Booking("BK001", customer, room2)
```

Execution Output

Console Resolution

```
# Accessing traversed properties:
```

```
print(booking.customer.name)
```

```
# Console Output:
```

```
John
```

```
print(booking.room.number)
```

```
# Console Output:
```

```
102
```

◇ References resolve dynamically across independent aggregate lifecycles

Why Use Aggregation?

Key Advantages

- **Reuse existing objects:** Avoid redundant instantiation by referencing pre-existing entities.
- **Easier to maintain:** Decoupled classes isolate codebase changes and prevent system-wide regression.
- **Flexible architectures:** Swap, reassign, or alter child references dynamically at runtime.
- **Independent lifecycles:** Keeps objects alive outside their container context.

Real-World Behavior

- **Room Persistence:**
A **Room** object continues to exist within the system memory even if its corresponding **Booking** instance is deleted or canceled.
- **Customer Persistence:**
A **Customer** object remains active and unaffected in the database long after checkout and booking destruction.

◇ Aggregation (HAS-A) ensures clean lifecycle isolation

Aggregation vs Inheritance

Inheritance (IS-A)

Relationship: Represents a strict hierarchy where a subclass is a specialized version of a parent class.

Real-World: Dog IS-A Animal. Employee IS-A Person.

```
class Child(Parent):
```

Design Guideline: Achieves code reuse by inheriting attributes and methods. Use when you can clearly say "IS A".

Aggregation (HAS-A)

Relationship: Represents a dynamic association where one class has or configures references to independent objects.

Real-World: Hotel HAS-A Rooms. Booking HAS-A Customer.

```
self.customer = customer
```

Design Guideline: Achieves object collaboration through memory references. Use when you can clearly say "HAS A".

Exercise: Hotel System

Requirements

Build a simple Hotel Booking System by implementing a clean object-oriented architecture in Python.

Create these classes:

- **Hotel**: Tracks hotel identity and rooms
- **Room**: Represents room details
- **Customer**: Represents guest information
- **Booking**: Manages reservation state

Ensure proper parameter initialization within each constructor.

Relationships

Establish correct object collaboration by passing references during instantiation.

Structural Architecture:

Hotel HAS MANY Room

└─ Aggregation (list of Room objects)

Booking HAS A Customer

Booking HAS A Room

```
# Avoid hardcoded attributes in Booking;  
# pass active instances as parameters.
```

Challenge: Class Designs

Room

Attributes

- `room_number`
- `room_type`
- `price`

Hotel

Attributes

- `hotel_name`
- `rooms`

Methods

- `add_room()`
- `display_rooms()`

Customer

Attributes

- `customer_name`
- `phone`

Booking

Attributes

- `booking_id`
- `customer`
- `room`

Methods

- `display_booking()`

Execution Output: Verification

Hotel State

Hotel Name: Paradise Hotel

Available Rooms:

101 | Standard | RM120

102 | Deluxe | RM180

Booking Information

Booking ID: BK001

Customer: Alice

Assigned Room: 102 (Deluxe)

Total Price: RM180

Bonus Challenge

Room Management Features

Extend your existing class design to include dynamic operational capabilities for controlling inventory:

◇ `search_room(number)`

Locates and returns details of a specific room instance.

◇ `remove_room(number)`

Deletes a room object from the hotel's collective database.

◇ `check_availability()`

Filters and returns lists of completely unbooked rooms.

◇ `display_booked_rooms()`

Quickly reviews only the rooms currently active in bookings.

Booking & Customer Features

Introduce complex memory reference management to support historical records and user listings:

◇ `book_multiple_rooms()`

Aggregates multiple Room object pointers within a single transaction.

◇ `cancel_booking(id)`

Purges booking instances and releases rooms back to inventory.

◇ `booking_history`

Retains logs of past and current associations in system memory.

◇ `display_all_customers()`

Iterates and lists unique Customer identities registered in system.

Think Like an Object-Oriented Programmer

Is this an **IS-A** relationship?

Use Inheritance

When one class is a specialized version of another class.

Real-World Example:

- Teacher IS A Person
- The child class inherits all attributes and methods from the parent class.

Is this a **HAS-A** relationship?

Use Aggregation

When a class obtains references to other independent instances.

Real-World Examples:

- Hotel HAS Rooms
- Booking HAS A Customer
- Booking HAS A Room

Rule of Thumb: Choosing the Correct Python OOP Design

Inheritance represents "**IS A**" relationships, whereas Aggregation model "**HAS A**" associations between active object instances.